

LifeMaps — AWS migration framework

What this document is: the runbook to move **hosting** from Render to AWS.

What it is not: a product rewrite, a GitHub move, or a database redesign.

Model (this is the whole move):

today: GitHub main → Render (app + API + optional classifier + Postgres) target: GitHub main → AWS (app + API + optional classifier + RDS)

The repo stays <https://github.com/Amitr16/lifemaps.git>, branch main. Developers keep pushing the same way. Only the **downstream host** changes. Existing users and plans move with a Postgres dump/restore. The other company website on the same AWS account is untouched except for a new hostname.

0. Decisions the AWS team must lock before work

Copy this table into the ticket and fill it. Do not start provision until all four are answered.

#	Decision	Options	Locked answer
1	Compute	Share existing EC2 or new small EC2/ECS in the same VPC	
2	Database	New RDS instance or new database name on existing Postgres	
3	Public hostname	e.g. <code>lifemap.example.com</code>	
4	TLS / edge	nginx + Let's Encrypt (same as other site) or ALB + ACM	
5	How AWS gets new commits	GitHub Actions deploy or git pull + restart on the box	
6	Classifier	Host it or leave it off (expense classify is optional)	
7	Freeze window	Date/time users should not save during final dump	

Recommended defaults if the other site is already nginx-on-EC2:

1. New small EC2 in the same VPC (isolation).
2. New RDS instance (do not share tables with the other site).
3. `lifemap.<company-domain>`.

4. Same TLS pattern as the other site.
5. GitHub Actions on push to `main` (same trigger Render uses today).
6. Classifier optional.
7. 15–30 minute freeze for the final dump.

1. Current production (Render)

Role	URL	Render service	How it starts
App	<code>https://lifemaps-frontend.onrender.com</code>	<code>lifemaps-frontend</code>	<code>npm run build</code> then <code>node server.js</code> (serves <code>dist/</code> , port 3000)
API	<code>https://lifemaps-backend.onrender.com</code>	<code>lifemaps-backend</code>	<code>cd backend && node start-render.js</code> → <code>node server.js</code> (port 10000)
Classifier	<code>https://lifemaps-classifier.onrender.com</code>	<code>lifemaps-classifier</code>	<code>python expense_classifier_service.py</code>
DB	internal	<code>lifemaps-db</code>	Managed Postgres

`render.yaml` in this repo is the blueprint. GitHub **auto-deploys** `main` to those services.

Frontend env (build-time): `VITE_API_URL=https://lifemaps-backend.onrender.com/api`

API env: `DATABASE_URL` (from `lifemaps-db`), `JWT_SECRET`,

`CORS_ORIGIN=https://lifemaps-frontend.onrender.com`, `CLASSIFIER_SERVICE_URL`, `PORT=10000`,

`NODE_ENV=production`.

Do not recreate or redeploy `lifemaps-db`. The init path drops tables.

2. What the product actually is (so AWS does not “fix” it)

Browser path	Renders
<code>/assets</code> <code>/work-assets</code> <code>/goals</code> <code>/loans</code> <code>/expenses</code>	React MockupHost + iframe of <code>/lifemap/*.html</code>
<code>/insurance</code> <code>/profile</code> <code>/growth-assumptions</code>	React inside Shell
<code>/admin/login</code> <code>/admin</code>	Admin
<code>/super-admin/login</code> <code>/super-admin</code>	Super admin

Admin is **not** in the mockup header. Use:

- `https://<app-host>/admin/login`
- `https://<app-host>/super-admin/login`
- or **Sign in** → small Admin / Super Admin links at the bottom of the modal

Signed-in data is **Postgres rows**, not a JSON dump and not localStorage (except a few growth-assumption extras in quickCalcAssumptions). Guest edits stay in the browser until register (register uploads; login loads the account and does not upload the demo).

Iframe files: `src/mockups/` → `python scripts/prepare-mockups.py` → `public/lifemap/` → Vite copies into `dist/`. If `/lifemap/fp-calculator.html` 404s, home is a blank iframe.

3. Target AWS shape

```
GitHub Amitr16/lifemaps main ■ ■ (GitHub Actions, or git pull on the box) ▼ Internet → Route 53 / existing DNS ■ ■■■ https://lifemap.example.com nginx (or ALB) / → static dist/ (SPA + mockup HTML) /lifemap/ → dist/lifemap/*.html /assets/*.js → Vite hashed bundles (trailing slash only) /api/ → 127.0.0.1:10000 /health → 127.0.0.1:10000/health systemd lifemaps-api node backend/server.js systemd lifemaps-clf optional Python :5001 RDS database lifemaps
```

Same-origin `/api` on the app host is the easiest CORS story. The SPA **bakes VITE_API_URL in at build time**. Changing the API URL later requires a rebuild.

LifeMaps has a quoted `"user"` table. **Never restore into the other website's database.**

There is no Redis, S3, SQS, or file upload. No IAM keys are required by the app.

4. Forbidden actions (read before any command)

Never	Why
<code>DATABASE_INIT=true</code>	<code>backend/start-render.js</code> then runs <code>scripts/init-render-db.js</code> , which DROP TABLE ... CASCADE
<code>node backend/scripts/init-render-db.js</code> against prod/RDS	Same drops
Recreate / restore-from-blueprint lifemaps-db on Render	Wipes live users
<code>git push --force</code> to main as part of this move	Unrelated to hosting
Point the other site's nginx <code>root</code> at LifeMaps <code>dist/</code>	Breaks the existing website
Restore the dump into the other site's RDS	Table name collision (<code>"user"</code>)

Put a <code>.env</code> in the web root	<code>backend/server.js</code> prints <code>.env</code> contents to stdout if the file exists
---	---

Schema on every API boot is **additive only**: `backend/scripts/ensure-lifemap-mockup-schema.js` (`ADD COLUMN IF NOT EXISTS, CREATE TABLE IF NOT EXISTS planned_loan`).

On AWS start the API with `node server.js`, not `node start-render.js`, so the destructive wrapper is not even in the process tree.

5. Phase plan

Phase	Name	Owner	Exit criteria
A	Discover	Product + AWS	Env copied, dump tested, decisions table filled
B	Provision	AWS	RDS empty, EC2/nginx/TLS, API /health against empty RDS
C	Rehearse	Both	Staging hostname: dump restored, UI smoke test pass
D	Freeze + cutover	Both	Final dump, DNS to AWS, production smoke test pass
E	Hypercare	Both	24–48h, Render still running but auto-deploy paused
F	Decommission Render	Product + AWS	Final Render dump archived, Render services deleted

Keep Render **live** through E. DNS is the switch.

Phase A — Discover (no AWS changes yet)

A1. Inventory from Render dashboard

Copy into a secrets store (1Password / SSM), not Slack:

- [] lifemaps-backend env: `DATABASE_URL` (External), `JWT_SECRET`, `CORS_ORIGIN`, `CLASSIFIER_SERVICE_URL`, `JWT_EXPIRES_IN` if set

- [] lifemaps-frontend env: VITE_API_URL
- [] lifemaps-classifier env: OPENAI_API_KEY (if classifier stays)
- [] Postgres version (match major on RDS)
- [] Confirm GitHub connected to Render on main

Copy JWT_SECRET if existing logins must keep working. A new secret signs everyone out (users just log in again; data is unharmed).

A2. Prove you can dump Render

From a laptop that can reach Render's **external** DB URL:

```
pg_dump "$RENDER_DATABASE_URL" --no-owner --no-acl -F c -f lifemaps.render.preflight.dump
pg_restore -l lifemaps.render.preflight.dump | head
```

Store the dump encrypted. This is a rehearsal dump, not the cutover dump.

A3. Row-count baseline (save this output)

```
SELECT 'user' AS t, count(*) FROM "user" UNION ALL SELECT 'financial_profile', count(*) FROM
financial_profile UNION ALL SELECT 'assets', count(*) FROM assets UNION ALL SELECT
'work_assets', count(*) FROM work_assets UNION ALL SELECT 'financial_loan', count(*) FROM
financial_loan UNION ALL SELECT 'planned_loan', count(*) FROM planned_loan UNION ALL SELECT
'financial_goal', count(*) FROM financial_goal UNION ALL SELECT 'financial_expense', count(*)
FROM financial_expense UNION ALL SELECT 'financial_insurance', count(*) FROM
financial_insurance UNION ALL SELECT 'admin', count(*) FROM admin UNION ALL SELECT
'super_admin', count(*) FROM super_admin;
```

planned_loan / admin tables may be empty; missing table means that feature never ran — boot migration will create planned_loan.

Phase B — Provision AWS (empty, Render still serving users)

B1. RDS

- PostgreSQL 14+ (match Render major if possible).
- New database lifemaps, new owner role.
- Same VPC as the compute.
- SG: inbound **5432 only from the LifeMaps compute SG**, not 0.0.0.0/0.
- Publicly inaccessible unless the company's jump-host pattern requires otherwise.
- Parameter: SSL required.

Connection string form the app expects:

```
postgres://USER:PASS@RDS_HOST:5432/lifemaps?sslmode=require
```

The Node client uses `DATABASE_URL` **with** `ssl: { rejectUnauthorized: false }` (Render-style). Split `DB_HOST` / `DB_PASSWORD` is the **local-dev path and has no SSL** — do not use it on RDS.

Optional on RDS before first restore:

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss"; CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

B2. Compute

New EC2 (or ECS) in the same VPC:

- Node 18+
- nginx (if not using ALB→Node directly)
- Python 3.11 only if classifier is in
- Disk enough for repo + node_modules + dist/ (2 GB is plenty)

Outbound: 443 (GitHub, npm; OpenAI if classifier), 5432 to RDS.

B3. Secrets on the box

```
sudo mkdir -p /etc/lifemaps sudo chmod 700 /etc/lifemaps
```

`/etc/lifemaps/api.env` (mode 0600):

```
NODE_ENV=production PORT=10000 DATABASE_URL=postgres://...@...:5432/lifemaps?sslmode=require
JWT_SECRET=<copied from Render> JWT_EXPIRES_IN=7d CORS_ORIGIN=https://lifemap.example.com
CLASSIFIER_SERVICE_URL=http://127.0.0.1:5001 RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=2000 MAX_CONCURRENT_REQUESTS=50
```

Leave `DATABASE_INIT` **unset**.

`/etc/lifemaps/classifier.env` only if hosting classifier:

```
CLASSIFIER_PORT=5001 DATABASE_URL=<same as API> OPENAI_API_KEY=<copied from Render>
```

Do not put these files in the git checkout.

B4. First API boot against empty RDS

```
sudo mkdir -p /var/www/lifemap sudo git clone https://github.com/Amitr16/lifemaps.git
/var/www/lifemap cd /var/www/lifemap sudo git checkout main cd backend sudo npm ci --omit=dev #
EnvironmentFile in systemd; then: sudo systemctl enable --now lifemaps-api curl -sS
https://127.0.0.1:10000/health # or via nginx once TLS exists
```

Logs must show schema helper applied, **not** “Dropping existing tables”. `GET /health` → `{ "status": "OK", ... }`.

systemd unit:

```
# /etc/systemd/system/lifemaps-api.service [Unit] Description=LifeMaps API After=network.target
[Service] Type=simple User=www-data WorkingDirectory=/var/www/lifemap/backend
EnvironmentFile=/etc/lifemaps/api.env ExecStart=/usr/bin/node server.js Restart=always
RestartSec=5 [Install] WantedBy=multi-user.target
```

Optional classifier unit: WorkingDirectory=/var/www/lifemap, ExecStart=/usr/bin/python3
expense_classifier_service.py, EnvironmentFile=/etc/lifemaps/classifier.env.

B5. nginx vhost (new file only)

Do **not** edit the other site's server { }.

```
# /etc/nginx/sites-available/lifemap server { listen 443 ssl http2; server_name
lifemap.example.com; ssl_certificate /etc/letsencrypt/live/lifemap.example.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/lifemap.example.com/privkey.pem; client_max_body_size
1m; root /var/www/lifemap/dist; index index.html; location /lifemap/ { try_files $uri =404; } #
Vite hashed JS/CSS. Trailing slash is required. # `location /assets` (no slash) would steal the
React route /assets. location /assets/ { try_files $uri =404; add_header Cache-Control "public,
max-age=31536000, immutable"; } location /health { proxy_pass http://127.0.0.1:10000/health;
proxy_http_version 1.1; proxy_set_header Host $host; proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto $scheme; } location /api/ {
proxy_pass http://127.0.0.1:10000/api/; proxy_http_version 1.1; proxy_set_header Host $host;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto
$scheme; proxy_set_header Authorization $http_authorization; } location / { try_files $uri
$uri/ /index.html; } }
```

HTTP→HTTPS redirect as you do for the other site. SPA routes that must hit index.html: /, /assets,
/work-assets, /goals, /loans, /expenses, /insurance, /profile, /growth-assumptions, /admin, /admin/login,
/super-admin, /super-admin/login.

B6. Frontend build (staging hostname first is fine)

```
cd /var/www/lifemap npm ci VITE_API_URL=https://lifemap.example.com/api npm run build # output:
./dist - nginx root
```

Until public DNS exists, use a staging name (lifemap-staging.example.com) in **both** VITE_API_URL and
CORS_ORIGIN, then rebuild for production names at cutover.

Phase C — Rehearse (staging DNS or hosts file)

1. Restore the **preflight** dump onto RDS (see Phase D commands).
2. Restart API. Confirm additive migration, not drops.
3. Compare row counts to A3.
4. Open the staging hostname and run **\$9 Acceptance**.
5. Throw away the staging restore if you will do a clean restore at cutover, **or** keep it and do a second dump that overwrites.

If restore fails on extensions or ownership, fix RDS and repeat. Do not invent a new schema.

Phase D — Freeze and cutover

D1. Freeze

Announce: do not Save / register for ~15–30 minutes. Pause is for dump consistency, not code freeze.

Optional: on Render backend, scale to 0 **after** dump if you can; otherwise dump while live and accept seconds of drift.

D2. Final dump and restore

```
# From a host that can see Render external DB and RDS pg_dump "$RENDER_DATABASE_URL" --no-owner
--no-acl -F c -f lifemaps.render.cutover.dump # Empty RDS of rehearsal data if you restored in
Phase C # (only if this RDS has never served public users) dropdb --if-exists lifemaps createdb
lifemaps # re-grant owner, re-create extensions pg_restore --no-owner --no-acl --exit-on-error
-d "$AWS_DATABASE_URL" lifemaps.render.cutover.dump
```

If `pg_restore` errors on existing objects from the empty-API boot, restore into a **fresh** database or drop/create `lifemaps` once (this RDS is not public yet).

Restart API. Re-run A3 counts on RDS; they must match the dump.

D3. Production frontend build

```
cd /var/www/lifemap git fetch origin && git checkout main && git pull --ff-only cd backend &&
npm ci --omit=dev && cd .. npm ci VITE_API_URL=https://lifemap.example.com/api npm run build
sudo systemctl restart lifemaps-api sudo nginx -t && sudo systemctl reload nginx
```

Set `CORS_ORIGIN` to the **exact** public origin (`https://lifemap.example.com`, no trailing slash). Rebuild if the API URL changed.

D4. DNS

Point `lifemap.example.com` at the ALB / EC2 the same way the other site does (A / ALIAS / CNAME). Leave Render URLs working.

TTL: lower to 60s an hour before cutover if you can.

D5. Go / no-go

- [] `/health` on the new host is OK
- [] `/lifemap/fp-calculator.html` is 200
- [] Login with an existing user (if `JWT_SECRET` was copied)
- [] §9 Acceptance on the **public** hostname
- [] No CORS errors in browser / API logs

If no-go: leave DNS on Render (or revert the record). RDS can stay; it is not public until DNS says so.

Phase E — Hypercare (24–48 hours)

- Render auto-deploy: **pause** (Settings → disconnect GitHub or pause service) so a push to `main` does not update Render while AWS is live.
- Watch API logs: CORS, 5xx, SSL to RDS.
- Product owner smoke-tests a real account Save.

GitHub `main` now deploys **only** to AWS (once §7 wiring is in).

Phase F — Decommission Render

- ☐ Final `pg_dump` of Render archived encrypted
- ☐ Confirm AWS has been stable
- ☐ Delete Render web services (`lifemaps-frontend`, `lifemaps-backend`, `lifemaps-classifier`)
- ☐ Delete Render DB **last**, after the dump is verified restorable

Do not delete the GitHub repo. Do not change `main` history for this step.

6. How GitHub keeps deploying (replace Render's GitHub integration)

Pick one.

Option 1 — GitHub Actions (recommended)

Add later (not required to stand the box up). On push to `main`: SSH or SSM to the instance, `git pull --ff-only`, `npm ci`, `backend npm ci`, `VITE_API_URL=... npm run build`, `systemctl restart lifemaps-api`, `nginx reload`.

Secrets in GitHub: `host`, `user`, `key`, `VITE_API_URL`. Do **not** put `DATABASE_URL` in Actions if the box already has `/etc/lifemaps/api.env`.

Option 2 — Pull on the box

Webhook or a 5-minute cron:

```
cd /var/www/lifemap && git pull --ff-only && \ (cd backend && npm ci --omit=dev) && \ npm ci &&
VITE_API_URL=https://lifemap.example.com/api npm run build && \ sudo systemctl restart
lifemaps-api
```

Until one of these exists, AWS will **not** follow `main` automatically. That is OK for first cutover; do not leave it that way.

Render's GitHub auto-deploy must be paused in Phase E or every push updates **both** hosts.

7. Environment matrix

API (backend/server.js)

Variable	Required	Production value
NODE_ENV	yes	production
PORT	yes	10000
DATABASE_URL	yes	RDS URL + sslmode=require
JWT_SECRET	yes	Copy from Render to keep sessions
JWT_EXPIRES_IN	no	7d
CORS_ORIGIN	yes	Exact browser origin(s), comma-separated
CLASSIFIER_SERVICE_URL	no	http://127.0.0.1:5001
RATE_LIMIT_WINDOW_MS	no	900000
RATE_LIMIT_MAX_REQUESTS	no	2000
MAX_CONCURRENT_REQUESTS	no	50
DATABASE_INIT	never	unset

Frontend (Vite build-time)

Variable	Required	Production value
VITE_API_URL	yes	https://lifemap.example.com/api

Classifier (optional)

Variable	Required
CLASSIFIER_PORT	5001
DATABASE_URL	same RDS
OPENAI_API_KEY	from Render

Health:

- GET `https://lifemap.example.com/health` → API
- GET `https://lifemap.example.com/api/...` → financial routes need JWT
- GET `http://127.0.0.1:5001/health` → classifier

8. Database notes

After restore, API boot adds missing mockup columns if the dump is older:

- `financial_profile`: `inflation_rate`, `equity_growth_rate`, `debt_growth_rate`, `personal_asset_value`
- `assets`: `category`, `sip_*`, `expected_return`, `notes` (`custom_data` already existed)
- `goals` / `expenses` / `loans` extra fields
- table `planned_loan`

Confirm:

```
SELECT table_name FROM information_schema.tables WHERE table_schema = 'public' ORDER BY 1;
SELECT column_name FROM information_schema.columns WHERE table_name = 'financial_profile' AND
column_name IN ('inflation_rate', 'equity_growth_rate', 'personal_asset_value'); SELECT
column_name FROM information_schema.columns WHERE table_name = 'assets' AND column_name IN
('sip_amount', 'sip_frequency', 'custom_data');
```

Expect at least: `"user"`, `financial_profile`, `assets`, `work_assets`, `financial_loan`, `planned_loan`, `financial_goal`, `financial_expense`, `financial_insurance`, plus `admin` / `super_admin` if they existed on Render.

Earmarks live in `assets.custom_data` / `financial_goal.custom_data`. Loan EMI rows use `financial_expense.loan_id`. Do not strip `custom_data` or those FKs.

9. Acceptance (staging and production)

Must all pass:

1. `/` shows LifeMap mockup (FP Calculator), not a blank iframe and not the old Life Sheet form.
2. `/lifemap/fp-calculator.html` and `/lifemap/assets.html` return 200.
3. `/assets` (the Assets **page**) renders the mockup, not a 404 from nginx.
4. Guest can edit; **Save my plan** opens register; after register, numbers persist.
5. Log out, log in: account hydrates; demo data does not overwrite.
6. Save a row on Assets, Work Assets, Goals, Loans (current + planned), Expenses, FP Calculator.
7. Loan with EMI **and** end year shows as a locked EMI on Expenses; Remove on Expenses does not delete the loan.
8. Earmark an Investment asset to a goal (and reverse on Goals). Personal assets are not used for goals.
9. Insurance tab opens the React insurance page.

10. `/admin/login` and `/super-admin/login` open (or Sign in modal footer links).
11. `/health` is OK.
12. Browser console: no CORS failures.

10. Rollback

Render stays up until Phase F.

Situation	Action
AWS UI/API broken before DNS change	Fix AWS; users still on Render
DNS already on AWS, AWS broken	Point DNS back to Render
Users wrote data on AWS after DNS	<code>pg_dump</code> RDS before switching DNS back; restore to Render only if you must keep those writes
Accidental table drop	Restore <code>lifemaps.render.cutover.dump</code> (or later RDS dump) onto a new database; never run <code>init-render-db</code>

11. Security / IAM

- EC2/ECS: outbound 443, outbound 5432 to RDS.
- RDS: inbound 5432 only from that SG.
- ALB/nginx: inbound 443.
- No AWS SDK in the app.
- Secrets: SSM or `/etc/lifemaps/*.env` mode 0600, not git.
- Tighten RDS SSL verification later if required; current client matches Render (`rejectUnauthorized: false`).

12. Ownership after the move

Item	Owner
GitHub <code>Amitr16/lifemaps main</code>	LifeMaps product
AWS VPC, EC2/ALB, nginx, TLS, RDS, DNS	Company AWS / DevOps
Render teardown	Product + AWS after hypercare

JWT_SECRET, OPENAI_API_KEY	Product (AWS stores the copy)
Schema	Additive SQL in backend/scripts/ensure-lifemap-mockup-schema.js; deploy API to apply

13. Troubleshooting

Symptom	Likely cause
Blank home page	/lifemap/*.html 404 — dist/ missing public/ copy, or nginx root wrong
/assets 404	nginx location /assets without trailing slash stole the SPA route
Login works on Render, 401 on AWS	Different JWT_SECRET
Browser CORS errors	CORS_ORIGIN ≠ exact https://host (scheme + host, no path)
API talks to DB without SSL	Using DB_HOST instead of DATABASE_URL
Empty plans after restore	Restored into the wrong database, or ran init-render-db
Save works then data vanishes	Frontend still built with old VITE_API_URL (still hitting Render)
Push to GitHub updates Render not AWS	Render auto-deploy still on; AWS deploy hook not wired

14. One-sitting cutover checklist

Use after Phases A–C are done.

- ☐ Freeze announced
- ☐ Final Render dump saved encrypted
- ☐ Restore to RDS; row counts match
- ☐ API restarted with node server.js (no DATABASE_INIT)
- ☐ Frontend rebuilt with production VITE_API_URL
- ☐ nginx reload; TLS valid
- ☐ DNS pointed at AWS
- ☐ §9 pass on public hostname

- ☐ Render auto-deploy paused
- ☐ Hypercare watch 24–48h
- ☐ Render dump archived; Render services deleted

GitHub never moved. Only the host behind `main` did.